

POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY
Faculty of Information Technology

Intelligent System Development

ASSIGNMENT 06

Recurrent Neural Networks on Time Series:
From Scratch, in Keras and in PyTorch

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Lưu Anh Dũng
Student ID: B23DCDK036
Semester: I.2025 – 2026
Submission: Individual Project

Hanoi, 2026

Contents

1	Overview	2
2	Theoretical Background of Recurrent Networks	3
2.1	Sequences and the Forecasting Problem	3
2.2	Mathematical Operators	3
2.3	The Recurrent Cell	4
2.4	Training a Recurrent Network	4
2.5	Vanishing and Exploding Gradients	5
2.6	Gated Cells: LSTM and GRU	5
2.7	Evaluating a Forecast	6
2.8	The Same Layer in Three Implementations	6
3	Data: Sources, Features and Splits	6
3.1	EC: Order Delivery on the Olist Marketplace	6
3.2	ST: Daily Trading Range of NYSE Stocks	7
3.3	Splitting, Scaling and Protocol	8
4	Model Architecture and Parameters	8
5	Implementation Details	9
5.1	NumPy from Scratch	9
5.2	Keras	9
5.3	PyTorch	10
5.4	Verification	10
6	Experiment 1: Order Delivery Time (EC)	10
7	Experiment 2: Next-Day Trading Range (ST)	11
8	Comparison of Implementations and Datasets	13
9	Conclusions and Limitations	13

1 Overview

This report studies the recurrent neural network (RNN) and implements the same model three times: **from scratch** in NumPy (**SC**: hand-written forward pass, back-propagation through time and Adam), in **Keras** (**TF**) and in **PyTorch** (**PT**). It is applied to two time-related datasets from Kaggle, each between 100 and 150 MB:

- **EC — a service process in e-commerce.** The Olist marketplace (Brazil, 2016–2018): from the last 10 orders sent to the same state, predict how many days the newest order will take to be delivered (96,227 windows).
- **ST — stock prices.** 250 NYSE stocks (2010–2016): from the last 30 trading days, predict the next day’s trading range, a measure of risk (432,750 windows).

Both tasks are many-to-one regression. All implementations share the data, the split, the architecture (one vanilla RNN layer with 32 units and a linear head), the starting weights, the optimizer, the early-stopping rule and the hardware (CPU), so only the abstraction level changes. LSTM and GRU are added in Keras and PyTorch. Run IDs have the form `<dataset>-<implementation>-<model>`, e.g. `ST-TF-GRU`.

Table 1: All 14 runs, test period. RMSE in days (EC, top) and in percentage points of daily range (ST, bottom); R^2 is out-of-sample against the train mean; s/epoch = median time per epoch on CPU.

Run	test RMSE	R^2	params	s/epoch
EC-SC-RNN	5.014 d	0.543	1,409	0.2
EC-TF-RNN	4.968 d	0.557	1,409	0.6
EC-PT-RNN	4.916 d	0.549	1,409	0.8
EC-TF-LSTM	4.916 d	0.563	5,537	1.3
EC-PT-LSTM	4.829 d	0.573	5,665	1.8
EC-TF-GRU	4.869 d	0.579	4,257	1.5
EC-PT-GRU	4.821 d	0.574	4,257	1.5
ST-SC-RNN	1.1897 pp	0.522	1,313	2.3
ST-TF-RNN	1.1894 pp	0.529	1,313	5.5
ST-PT-RNN	1.1842 pp	0.529	1,313	8.0
ST-TF-LSTM	1.1559 pp	0.537	5,153	14.8
ST-PT-LSTM	1.1454 pp	0.534	5,281	22.1
ST-TF-GRU	1.1606 pp	0.537	3,969	17.3
ST-PT-GRU	1.1486 pp	0.533	3,969	17.9

Key takeaways

1. **The three implementations compute the same function.** With identical weights the forward passes differ by less than $5 \cdot 10^{-7}$ and the parameter counts are equal (1,409 for EC, 1,313 for ST). After training the core RNN ends within 2 % (EC) and 0.5 % (ST) across implementations; the spread comes from batch order and the stopping epoch, not from the mathematics.
2. **On EC the sequence helps.** Every recurrent model beats every baseline: RMSE 4.82–5.01 days against 5.22 for last week’s mean delivery time to the state, 5.26 for a linear model on the same inputs and 5.41 for Olist’s recalibrated promise (R^2 0.54–0.58 vs. at most 0.48).
3. **On ST risk is predictable, but the classic linear HAR model is as good.** The RNNs reach $R^2 = 0.52$ – 0.54 and call the direction of tomorrow’s change in risk correctly 70.5–71.1 % of the time, far above naive forecasts; the best (PyTorch LSTM) only ties HAR (1.1454 vs. 1.1449 pp).

4. **Gates help moderately.** LSTM/GRU are the best models on both datasets (1–3 % lower RMSE than the vanilla RNN), at $2\text{--}3\times$ the time per epoch and $3\text{--}4\times$ the parameters. The vanilla RNN passes only 2 % (EC) and 6 % (ST) of the learning signal to the oldest step of its window.
5. **Cost.** For the vanilla RNN on CPU the NumPy loop is the fastest per epoch (EC 0.24 s, ST 2.3 s) ahead of Keras (0.6, 5.5 s) and PyTorch (0.8, 8.0 s), because the model is tiny and the frameworks carry fixed overhead per batch.

2 Theoretical Background of Recurrent Networks

This chapter develops what is needed to read the rest of the report: the forecasting task (2.1), the operators (2.2), the recurrent cell (2.3), training (2.4), vanishing and exploding gradients (2.5), gated cells (2.6), evaluation (2.7) and the same layer in three libraries (2.8). Every numeric example was checked in the concepts notebook against NumPy, finite differences or PyTorch. Notation: N windows, T window length, F features per step, H hidden size; $x_t \in \mathbb{R}^F$, $h_t \in \mathbb{R}^H$, prediction $\hat{y}$, target y , loss L . Vectors are row vectors, so a layer is written xW .

2.1 Sequences and the Forecasting Problem

Order is information. A dense network sees one vector and has no notion of which input came first. In a sequence the order matters: delivery times of 8, 12, 15 days (logistics getting worse) and 15, 12, 8 (recovering) have the same mean but call for different forecasts.

Windows. A series $s_1, \dots, s_n$ becomes supervised pairs by sliding a window of T steps: input $(s_k, \dots, s_{k+T-1})$, target s_{k+T} . For the delivery times 8, 12, 9, 15, 7, 21, 10 and $T = 3$: $[8, 12, 9] \rightarrow 15$, $[12, 9, 15] \rightarrow 7$, $[9, 15, 7] \rightarrow 21$, $[15, 7, 21] \rightarrow 10$. Stacked, inputs form a tensor $X \in \mathbb{R}^{N \times T \times F}$ — here $(4, 3, 1)$; in the EC experiment (96k, 10, 10), in ST (433k, 30, 7). Windows of many series (states, stocks) are stacked along N so that one model serves all of them.

Task shapes. *Many-to-one* reads T steps and outputs one value from the last state (used here); *many-to-many* outputs at every step; *one-to-many* generates a sequence. Only which states feed the output layer changes.

Time order in evaluation. The data are split in time: train on the past, validate and test on the future, and fit scaling statistics on the train period only. A random split would let the model learn from days after the ones it is tested on.

2.2 Mathematical Operators

- **Matrix product** $(xW)_j = \sum_i x_i W_{ij}$ mixes inputs and memory into H new values: $(2, 1) \begin{pmatrix} 1 & -1 \\ 0.5 & 2 \end{pmatrix} = (2.5, 0)$.
- **Bias** shifts each unit: $(2.5, 0) + (-0.5, 0.1) = (2, 0.1)$.
- **Hyperbolic tangent** $\tanh z \in (-1, 1)$, $\tanh'(z) = 1 - \tanh^2 z$: $\tanh 2 = 0.964$ and $\tanh'(2) = 0.071$ — a saturated unit passes 7 % of a gradient.
- **Sigmoid** $\sigma(z) = 1/(1 + e^{-z}) \in (0, 1)$, $\sigma' = \sigma(1 - \sigma) \leq 0.25$; used for gates: $\sigma(3) = 0.953$.
- **Element-wise (Hadamard) product** $(a \odot b)_i = a_i b_i$ applies a gate to a value: $(0.95, 0.1) \odot (2, 2) = (1.9, 0.2)$.
- **Softmax** $e^{z_i} / \sum_j e^{z_j}$ turns scores into probabilities, $(2, 1, 0) \mapsto (0.665, 0.245, 0.090)$; needed for classification, not for the regression heads used here.

- **Concatenation.** $x_t W_{xh} + h_{t-1} W_{hh} = [x_t, h_{t-1}] \begin{bmatrix} W_{xh} \\ W_{hh} \end{bmatrix}$: frameworks compute one larger product; the result is identical.

2.3 The Recurrent Cell

The network keeps a state $h_t \in \mathbb{R}^H$ that summarises everything read so far:

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b), \quad h_0 = 0, \quad \hat{y} = h_T W_{hy} + b_y, \quad (1)$$

with $W_{xh} \in \mathbb{R}^{F \times H}$, $W_{hh} \in \mathbb{R}^{H \times H}$, $b \in \mathbb{R}^H$, $W_{hy} \in \mathbb{R}^{H \times 1}$. *Unrolling* draws the loop as T copies of one cell (Figure 1). The copies *share* their weights, so the parameter count $FH + H^2 + H$ (cell) $+ H + 1$ (head) does not depend on T : the 10-step EC cell and the 30-step ST cell have the same size for equal F .

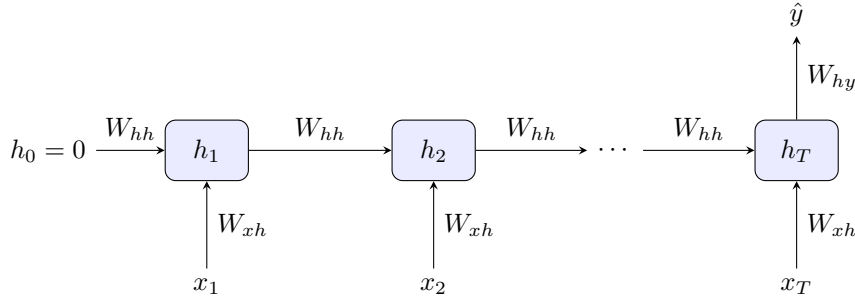


Figure 1: A many-to-one RNN unrolled over T steps; the same W_{xh} , W_{hh} , b are used at every step and only h_T feeds the output.

Worked example ($F = 1$, $H = 2$, $b = 0$). Daily ranges 1.2, 1.1, 2.9 %, centred by subtracting 1.5: $x = (-0.3, -0.4, 1.4)$, with $W_{xh} = (0.8, -0.5)$ and $W_{hh} = \begin{pmatrix} 0.5 & 0 \\ 0.2 & 0.3 \end{pmatrix}$:

t	x_t	$z_t = x_t W_{xh} + h_{t-1} W_{hh}$	$h_t = \tanh z_t$
1	-0.3	(-0.24, 0.15)	(-0.2355, 0.1489)
2	-0.4	(-0.4080, 0.2447)	(-0.3867, 0.2399)
3	1.4	(0.9746, -0.6280)	(0.7507, -0.5567)

Unit 1 tracks “risk is high now”; the two calm days still enter z_3 through $h_2 W_{hh}$. PyTorch’s `nn.RNN` with the same weights gives the same h_3 to 10^{-6} .

Memory. For one unit in its near-linear range, a small input spike decays as $h_t \approx w^{t-1} h_1$ for recurrent weight w : with $w = 0.9$ the spike keeps $0.9^9 = 39\%$ after 10 steps, with $w = 0.5$ only 0.2 % (Figure 2, left). The recurrent weight sets the memory length.

2.4 Training a Recurrent Network

Back-propagation through time (BPTT). With $z_t = x_t W_{xh} + h_{t-1} W_{hh} + b$ and $L = \frac{1}{N} \sum (\hat{y} - y)^2$:

$$\frac{\partial L}{\partial h_T} = \frac{2}{N} (\hat{y} - y) W_{hy}^\top, \quad \frac{\partial L}{\partial z_t} = \frac{\partial L}{\partial h_t} \odot (1 - h_t^2), \quad \frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial z_t} W_{hh}^\top, \quad (2)$$

and each shared weight collects one contribution per step: $\partial L / \partial W_{xh} = \sum_t x_t^\top \partial L / \partial z_t$, $\partial L / \partial W_{hh} = \sum_t h_{t-1}^\top \partial L / \partial z_t$, $\partial L / \partial b = \sum_t \partial L / \partial z_t$.

Scalar example. $h_t = \tanh(w x_t + u h_{t-1})$, $\hat{y} = h_2$, $x = (1, 0.5)$, $w = 0.6$, $u = 0.9$, $y = 0$. Forward: $h_1 = \tanh 0.6 = 0.5370$, $h_2 = \tanh(0.3 + 0.9 \cdot 0.5370) = 0.6546$, $L = 0.4285$. Backward: $\partial L / \partial z_2 = 2 \cdot 0.6546 (1 - 0.6546^2) = 0.7482$; step-2 share of $\partial L / \partial u$ is $0.7482 \cdot h_1 = 0.4018$, step 1 adds

0 (since $h_0 = 0$); $\partial L/\partial z_1 = 0.7482 \cdot u(1 - h_1^2) = 0.4792$ and $\partial L/\partial w = 0.7482 \cdot 0.5 + 0.4792 \cdot 1 = 0.8533$. PyTorch autograd returns the same 0.4018 and 0.8533.

Adam keeps running averages of the gradient and its square, $m \leftarrow \beta_1 m + (1 - \beta_1)g$, $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$, and steps $\theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$ with bias-corrected $\hat{m}, \hat{v}$; $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$, $\epsilon = 10^{-7}$.

Gradient clipping by global norm: $g \leftarrow g \cdot \min(1, \tau/\|g\|)$. With $g = (3, 4)$, $\|g\| = 5$, $\tau = 1$: $(0.6, 0.8)$ — direction kept, step length capped. Clipping each entry to $[-1, 1]$ instead would give $(1, 1)$ and change the direction.

Early stopping. After every epoch the validation loss is computed; training stops after 5 epochs without improvement and the best weights are restored.

2.5 Vanishing and Exploding Gradients

Repeating the last relation of Eq. (2), the signal reaching step t is a product of $T - t$ factors $\text{diag}(1 - h^2)W_{hh}^\top$. In the scalar case each step multiplies by $u(1 - h^2)$: a factor 0.5 leaves $0.5^{10} = 0.001$ after 10 steps and 10^{-9} after 30 (*vanishing*); a factor 1.3 gives $1.3^{30} \approx 2,620$ (*exploding*). Early inputs then cannot influence learning, or one update throws the weights away. Figure 2 (right) measures this on a 16-unit RNN over 30 steps for recurrent matrices of spectral radius 0.5–1.5. Clipping treats explosions; vanishing needs a different cell.

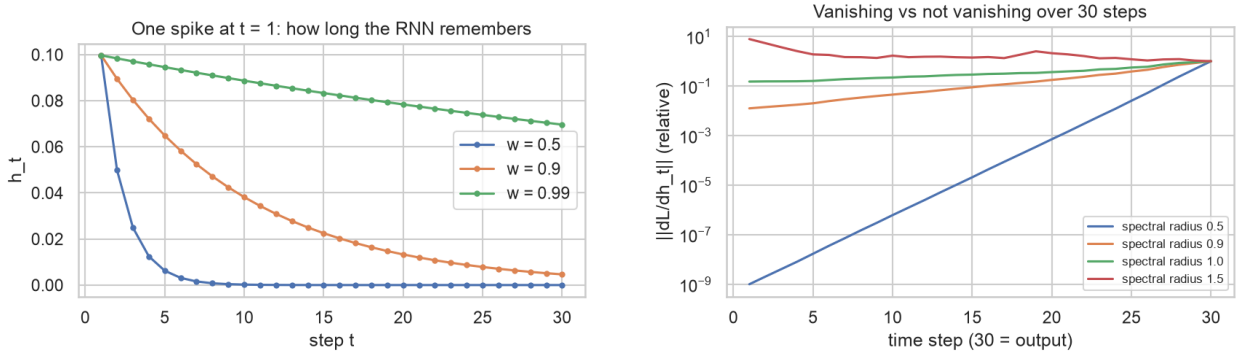


Figure 2: Left: response of a one-unit RNN to a single spike for three recurrent weights — the weight sets the memory length. Right: gradient norm reaching each of 30 steps, relative to the last step, for four spectral radii of W_{hh} .

2.6 Gated Cells: LSTM and GRU

A *gate* is a sigmoid vector multiplied element-wise onto a value; 0.95 keeps 95%, 0.05 nearly erases it, and the network learns when to do which.

GRU (one state): reset gate $r_t = \sigma(x_t W_r + h_{t-1} U_r + b_r)$, update gate $u_t = \sigma(x_t W_u + h_{t-1} U_u + b_u)$, candidate $\tilde{h}_t = \tanh(x_t W_h + r_t \odot (h_{t-1} U_h) + b_h)$ and

$$h_t = (1 - u_t) \odot \tilde{h}_t + u_t \odot h_{t-1}. \quad (3)$$

Example: $h_{t-1} = 0.8$, $u_t = 0.9$, $\tilde{h}_t = -0.5$ gives $h_t = 0.1 \cdot (-0.5) + 0.9 \cdot 0.8 = 0.67$ — the memory survives a contradicting input, and $\partial h_t / \partial h_{t-1} \approx u_t$ is an additive path that does not saturate. A hand-computed GRU step matches `nn.GRUCell` to 10^{-6} .

LSTM (cell c_t and output h_t): with input, forget and output gates i_t, f_t, o_t and candidate $\tilde{c}_t$,

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t). \quad (4)$$

With $f = 1$, $i = 0$ the cell is copied unchanged for any number of steps ($\partial c_t / \partial c_{t-1} = 1$); with $f = 0.9$ it decays as 0.9^t .

Parameters for input F and hidden H : RNN $H(F + H + 1)$; GRU $3H(F + H + 2)$ (two biases per block in Keras `reset_after=True` and PyTorch); LSTM $4H(F + H + 1)$ in Keras and $4H(F + H + 2)$ in PyTorch.

2.7 Evaluating a Forecast

A model is useful only if it beats simple forecasts, so each experiment has baselines (Section 3). On the test period:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum (y - \hat{y})^2}, \quad \text{MAE} = \frac{1}{N} \sum |y - \hat{y}|, \quad R_{\text{oos}}^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y}_{\text{train}})^2}, \quad (5)$$

where $R_{\text{oos}}^2 < 0$ means worse than predicting the train mean. RMSE and MAE are reported in original units (days, percentage points) after undoing the transforms; R_{oos}^2 is computed on the scale the models are trained on. EC also reports the share of predictions within ± 3 days; ST reports the *directional accuracy*, the share of days on which $\text{sign}(\hat{y}_{t+1} - y_t) = \text{sign}(y_{t+1} - y_t)$.

2.8 The Same Layer in Three Implementations

Table 2: The same recurrent layer in the three implementations.

	Scratch (NumPy)	Keras <code>SimpleRNN(H)</code>	PyTorch <code>nn.RNN</code>
Input	(N, T, F)	(N, T, F)	(N, T, F) with <code>batch_first=True</code>
W_{xh}	(F, H)	<code>kernel</code> (F, H)	<code>weight_ih_10</code> (H, F) , transposed
W_{hh}	(H, H)	<code>recurrent_kernel</code>	<code>weight_hh_10</code> , transposed
Bias	one vector	one vector	two vectors (<code>bias_ih</code> , <code>bias_hh</code>)
Output	h_T	h_T	all h_t ; take <code>out[:, -1]</code>
Backward pass	written by hand	<code>GradientTape</code> inside <code>fit</code>	<code>loss.backward()</code>
Training loop	written by hand	<code>model.fit</code>	written by hand

3 Data: Sources, Features and Splits

Both datasets are classic public Kaggle datasets of 100–150 MB, downloaded with `kagglehub`.

3.1 EC: Order Delivery on the Olist Marketplace

Source. *Brazilian E-Commerce Public Dataset by Olist* (`olistbr/brazilian-ecommerce`, 126 MB): 99,441 orders, 2016-09 to 2018-08, with every timestamp of the service process — purchase, payment approval, hand-over to the carrier, delivery — and the date promised to the customer.

Task. The 96,470 delivered orders are grouped by the customer’s state (27 states) and sorted by purchase time. A window is 10 consecutive orders to one state; its label is the delivery time, in days, of the last of them (bought Monday 10:00, delivered the next Thursday 16:00 $\Rightarrow$ 10.25 days). This gives 96,227 windows. **No look-ahead:** when an order is placed, earlier orders may still be travelling, so their delivery times are not inputs; every feature is known at purchase (Table 3). The feature `recent` carries past performance safely: it averages only deliveries completed before the purchase.

Table 3: EC features per order (all known at purchase time). The target is $\log(1 + \text{days})$, z-scored.

Feature	Meaning	Transform
<code>promise</code>	promised delivery date – purchase (days)	z
<code>price, freight, items</code>	order value, shipping fee (BRL), number of items	$\log(1 + x)$, z
<code>distance</code>	seller → customer great-circle distance from the zip-code coordinates	$\log(1 + \text{km})$, z
<code>same_state</code>	seller and customer in the same state	0/1
<code>recent</code>	mean delivery time of the state’s orders delivered in the 7 days before the purchase	z
<code>gap</code>	hours since the previous order to the same state	$\log(1 + x)$, z
<code>dow_sin, dow_cos</code>	weekday of purchase on a circle	–

What the data show (Figure 3). Delivery time is skewed (median 10.2 days, tail beyond 40), while Olist’s median promise is 23.2 days, so only 8.1 % of orders arrive late. The weekly median moves over time — up around Black Friday 2017 and the truckers’ strike of May 2018 — and successive orders to a state remain correlated (log-delivery autocorrelation 0.37 at lags 1 and 9), which is what a recurrent model can use. Distance explains a large part (correlation 0.55 on the log scale).

Baselines. Train mean; *recent state mean* (“as long as last week’s deliveries to this state”); *promise regression* (least squares on the promised days); and a *linear model* on the 100 flattened window values.

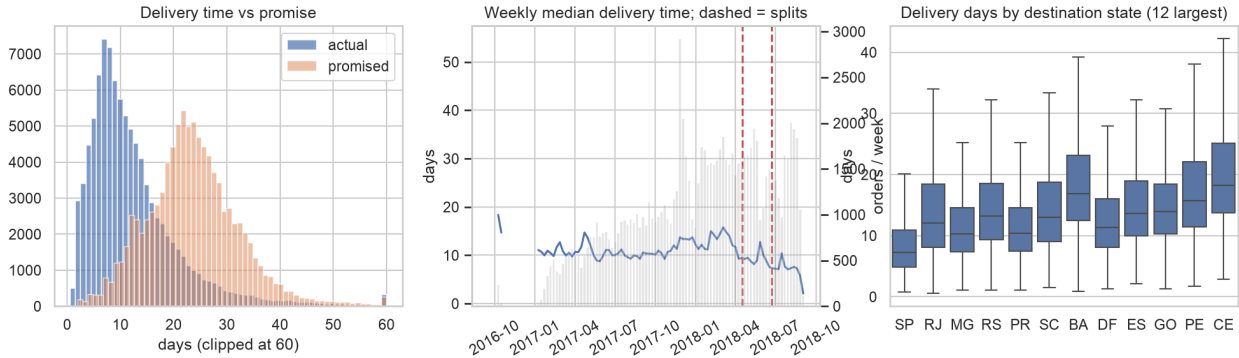


Figure 3: EC. Left: actual vs promised delivery days. Middle: weekly median delivery time (line) and orders per week (bars); dashed lines are the split boundaries. Right: delivery time by destination state.

3.2 ST: Daily Trading Range of NYSE Stocks

Source. *New York Stock Exchange* (dgawlik/nyse, 106 MB), file `prices-split-adjusted.csv`: daily open, high, low, close and volume of 501 companies, 2010-01-04 to 2016-12-30. The 250 most traded (by mean dollar volume) among the 467 with the full 1,762-day history are kept, giving $250 \times 1,731 = 432,750$ windows.

Why the range. A liquid stock’s next-day return is close to unpredictable; its next-day *risk* is not, because large moves cluster. The daily range $\ln(\text{high}/\text{low})$ (Parkinson, 1980) measures that risk: high 102, low 100 gives $\ln 1.02 = 0.0198 \approx 2\%$. The target is the log of the range on day $t + 1$, which makes it nearly Gaussian and turns differences between stocks into shifts ($2\% \rightarrow -3.91$, $4\% \rightarrow -3.22$).

Features (7 per day): `log_range`; log return $\ln(\text{close}_t/\text{close}_{t-1})$ and its absolute value; $\ln(\text{close}/\text{open})$; $\ln(1 + \text{volume})$ minus the stock’s train-period mean; weekday on a circle. Window $T = 30$ trading days.

What the data show (Figure 4). The log range is strongly autocorrelated (0.45 at lag 1, 0.22 at lag 22, 0.14 at lag 60) while returns are not (-0.02): risk has memory, returns do not. The test year contains the early-2016 sell-off and the Brexit-vote spike.

Baselines. Persistence (tomorrow = today), 5-day mean, 22-day mean, train mean, and **HAR** (Corsi, 2009): least squares on today’s value and the 5- and 22-day means, the standard linear model of volatility. Fitted coefficients: day 0.18, week 0.30, month 0.44.

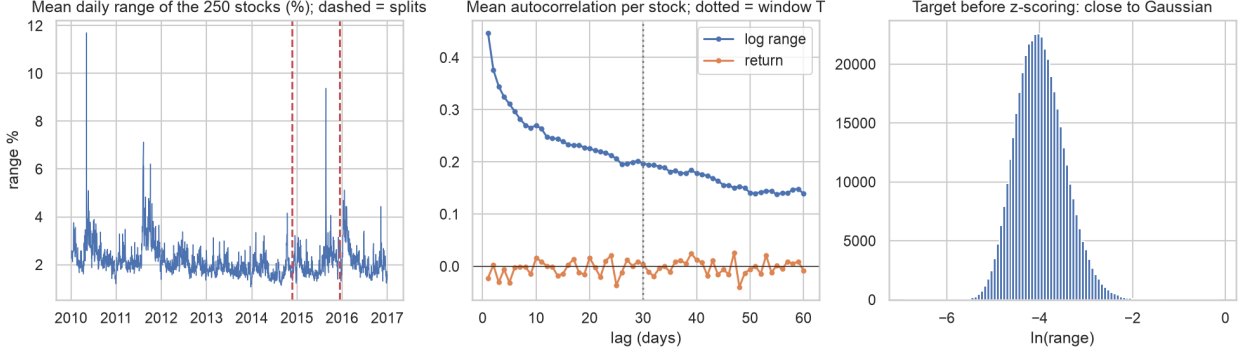


Figure 4: ST. Left: mean daily range of the 250 stocks with the split boundaries. Middle: mean autocorrelation of the log range and of the return; the dotted line is $T = 30$. Right: distribution of the log range.

3.3 Splitting, Scaling and Protocol

- **Chronological split 70/15/15** by the time of the predicted event. EC: purchase time, train before 2018-04-16, validation to 2018-06-21, test to 2018-08-29 (the truckers’ strike falls in validation). ST: calendar date, train before 2014-11-25, validation to 2015-12-14, test to 2016-12-30.
- Windows near a boundary look back into the earlier period; targets never cross it. Scaling statistics come from the train period only.
- Windows are built once; the identical arrays go to all three implementations. Seed 42 everywhere.

4 Model Architecture and Parameters

The core model, identical in the three implementations, is: input $(N, T, F) \rightarrow$ one vanilla RNN layer, $H = 32$, $\tanh \rightarrow h_T \rightarrow$ one linear unit, trained with MSE on the z-scored target. LSTM and GRU variants (one layer, $H = 32$) use each framework’s default initialisation.

Table 4: Hyperparameters (same for every run unless two values are given, EC / ST).

Name	Value	Meaning
Series	27 states / 250 stocks	pooled into one model
Window length T	10 orders / 30 days	steps read per prediction
Features F	10 / 7	Section 3
Windows (train / val. / test)	67,359 / 14,434 / 14,434 (EC); 300,500 / 66,000 / 66,250 (ST)	chronological 70/15/15
Hidden size H	32	dimension of h_t
Loss	MSE	on the z-scored target
Optimizer	Adam, $\eta = 10^{-3}$	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$
Batch size	128	reshuffled every epoch
Epochs	≤ 30 , early stopping patience 5	best weights restored
Gradient clipping	global norm 1.0	all three implementations
Initialisation (core)	Glorot W_{xh} , orthogonal W_{hh} , Glorot W_{hy} , zero biases	drawn once, copied to all
Hardware	CPU (Apple silicon)	Keras forced off the Metal GPU

Table 5: Trainable parameters. Core RNN identical in all implementations; PyTorch’s LSTM keeps two bias vectors per gate ($+4H = 128$).

Model	EC ($F = 10$)		ST ($F = 7$)	
	Keras	PyTorch	Keras	PyTorch
RNN ($FH + H^2 + H + \text{head } H + 1$)	1,409	1,409	1,313	1,313
GRU	4,257	4,257	3,969	3,969
LSTM	5,537	5,665	5,153	5,281

5 Implementation Details

5.1 NumPy from Scratch

The class `ScratchRNN` stores $W_{xh}, W_{hh}, b, W_{hy}, b_y$. The forward pass computes XW_{xh} for all steps in one product and loops only over time; the backward pass runs $t = T, \dots, 1$ through Eq. (2), accumulating the shared-weight gradients and recording $\|\partial L / \partial h_t\|$. Adam and global-norm clipping are hand-written.

Gradient check. Every parameter is checked against central differences ($\epsilon = 10^{-6}$, float64, 8 real windows); error = $\max |a - n| / \max |a|$:

Table 6: Gradient check of the scratch BPTT (threshold 10^{-5}).

Parameter	EC	ST
W_{xh}	$2.1 \cdot 10^{-10}$	$4.3 \cdot 10^{-10}$
W_{hh}	$8.2 \cdot 10^{-10}$	$6.4 \cdot 10^{-10}$
b	$5.7 \cdot 10^{-10}$	$4.5 \cdot 10^{-10}$
W_{hy}	$1.1 \cdot 10^{-10}$	$1.1 \cdot 10^{-10}$
b_y	$8.5 \cdot 10^{-12}$	$4.4 \cdot 10^{-12}$

5.2 Keras

`Sequential([Input((T,F)), SimpleRNN(32), Dense(1)])` with Adam (`global_clipnorm=1.0`) and MSE; fit with `EarlyStopping(restore_best_weights=True)`. The scratch weights are

loaded with `set_weights` in the same xW orientation. LSTM and GRU replace the recurrent layer.

5.3 PyTorch

`nn.RNN(F, 32, batch_first=True)` and `nn.Linear(32, 1)` on `out[:, -1]`; a hand-written loop runs `zero_grad` $\rightarrow$ `forward` $\rightarrow$ `backward` $\rightarrow$ `clip_grad_norm` $\rightarrow$ `step` and restores the best state. Weights are copied transposed and the second bias `bias_hh_10` is zeroed and frozen so the parameter count equals the other two.

5.4 Verification

With the same weights, 64 validation windows give forward outputs that differ by at most $4.8 \cdot 10^{-7}$ (EC) and $3.6 \cdot 10^{-7}$ (ST) between any two implementations — float32 round-off — and the parameter counts are equal. Differences after training therefore come from training randomness only.

6 Experiment 1: Order Delivery Time (EC)

Table 7: EC, test period. RMSE/MAE in days; ± 3 d = share of predictions within three days; ms/1k = inference time per 1,000 windows.

Run	RMSE (d)	MAE (d)	± 3 d (%)	R^2	params	epochs (best)	s/epoch	ms/1k
<i>train mean</i>	6.228	5.020	28.3	0.000	—	—	—	—
<i>recent state mean (last 7 days)</i>	5.215	3.444	56.6	0.472	—	—	—	—
<i>promise regression (OLS on promised days)</i>	5.408	3.682	53.8	0.453	—	—	—	—
<i>linear model on window (OLS)</i>	5.263	3.828	49.0	0.485	—	—	—	—
EC-SC-RNN	5.014	3.505	54.6	0.543	1,409	11 (6)	0.2	1.24
EC-TF-RNN	4.968	3.407	57.2	0.557	1,409	14 (9)	0.6	4.39
EC-PT-RNN	4.916	3.426	55.1	0.549	1,409	11 (6)	0.8	1.30
EC-TF-LSTM	4.916	3.363	57.8	0.563	5,537	7 (2)	1.3	6.49
EC-PT-LSTM	4.829	3.301	58.2	0.573	5,665	11 (6)	1.8	3.82
EC-TF-GRU	4.869	3.257	60.4	0.579	4,257	8 (3)	1.5	6.18
EC-PT-GRU	4.821	3.278	58.8	0.574	4,257	11 (6)	1.5	3.68

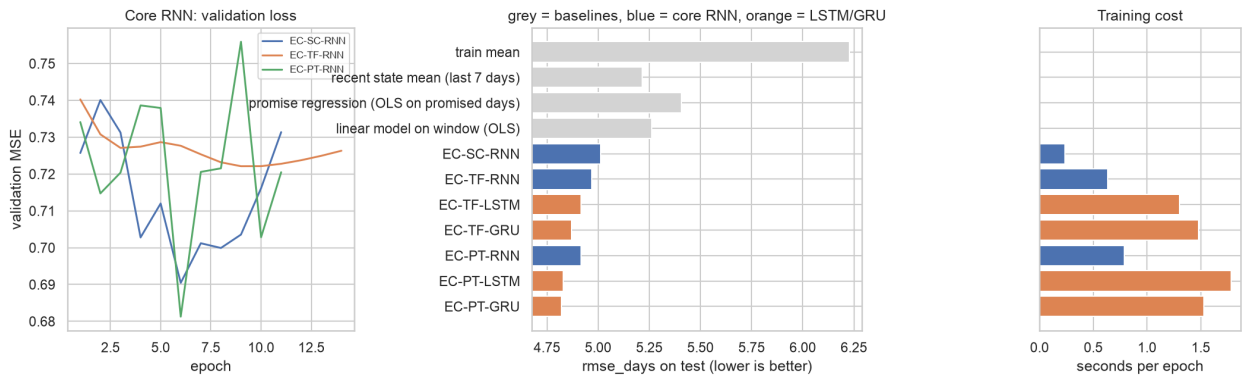


Figure 5: EC. Left: validation loss of the three core implementations. Middle: test RMSE (grey baselines, blue core RNN, orange LSTM/GRU). Right: time per epoch.

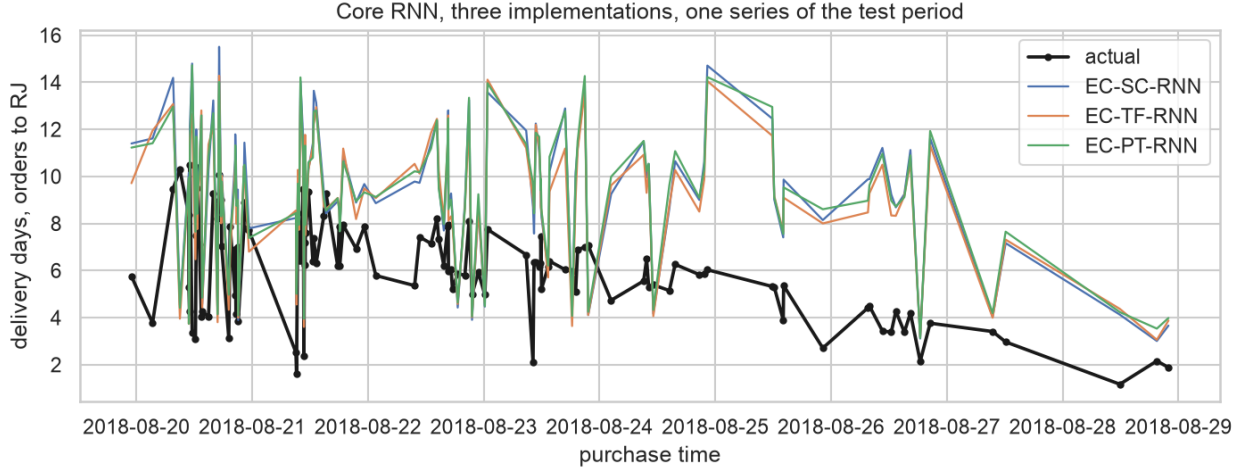


Figure 6: EC. Actual and predicted delivery times of the last 120 test orders to Rio de Janeiro, for the three core implementations.

Findings.

- All seven recurrent models beat all four baselines (RMSE 4.82–5.01 vs. ≥ 5.22 days). The linear model on exactly the same 100 numbers reaches only 5.26, so the gain comes from the non-linear, sequential combination of distance, congestion and calendar, not from extra information.
- Olist’s promise is a poor forecast even after recalibration (5.41 days): the operator’s own recent delivery speed to a state is more informative than the static promise.
- The three core implementations end at 5.01, 4.97 and 4.92 days. With only 67k training windows the stopping epoch varies (6, 9, 6), which explains the 2 % spread.
- Gates help: GRU and LSTM reach 4.82–4.92 days and the highest ± 3 -day hit rate (up to 60 %). The trained vanilla RNN passes only 1.8 % of the output gradient to the oldest of the 10 orders (Figure 9).

7 Experiment 2: Next-Day Trading Range (ST)

Table 8: ST, test period (2015-12-14 to 2016-12-30). RMSE/MAE in percentage points of daily range; dir. = directional accuracy.

Run	RMSE (pp)	MAE (pp)	R^2	dir. (%)	params	epochs (best)	s/epoch	ms/1k
<i>persistence (today)</i>	1.3852	0.8639	0.276	–	–	–	–	–
<i>5-day mean</i>	1.1631	0.7172	0.471	68.3	–	–	–	–
<i>22-day mean</i>	1.1888	0.7265	0.459	69.4	–	–	–	–
<i>train mean</i>	1.6528	0.9865	0.000	64.2	–	–	–	–
<i>HAR linear (day+week+month)</i>	1.1449	0.6820	0.520	70.0	–	–	–	–
ST-SC-RNN	1.1897	0.6893	0.522	70.5	1,313	8 (3)	2.3	2.72
ST-TF-RNN	1.1894	0.6821	0.529	70.8	1,313	8 (3)	5.5	6.19
ST-PT-RNN	1.1842	0.6828	0.529	70.7	1,313	8 (3)	8.0	3.86
ST-TF-LSTM	1.1559	0.6734	0.537	71.1	5,153	7 (2)	14.8	13.32
ST-PT-LSTM	1.1454	0.6765	0.534	71.0	5,281	7 (2)	22.1	11.68
ST-TF-GRU	1.1606	0.6742	0.537	71.1	3,969	6 (1)	17.3	14.89
ST-PT-GRU	1.1486	0.6765	0.533	70.8	3,969	7 (2)	17.9	11.06

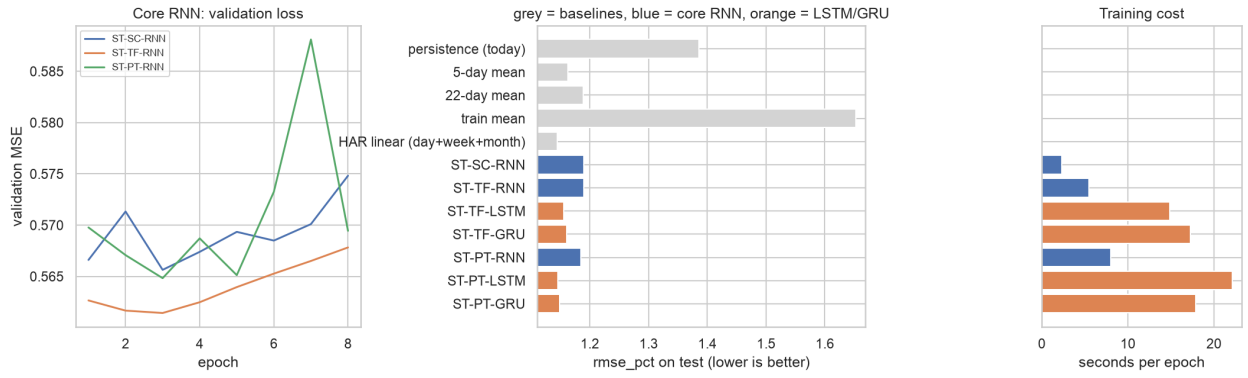


Figure 7: ST. Validation loss of the core implementations, test RMSE against the baselines and time per epoch.

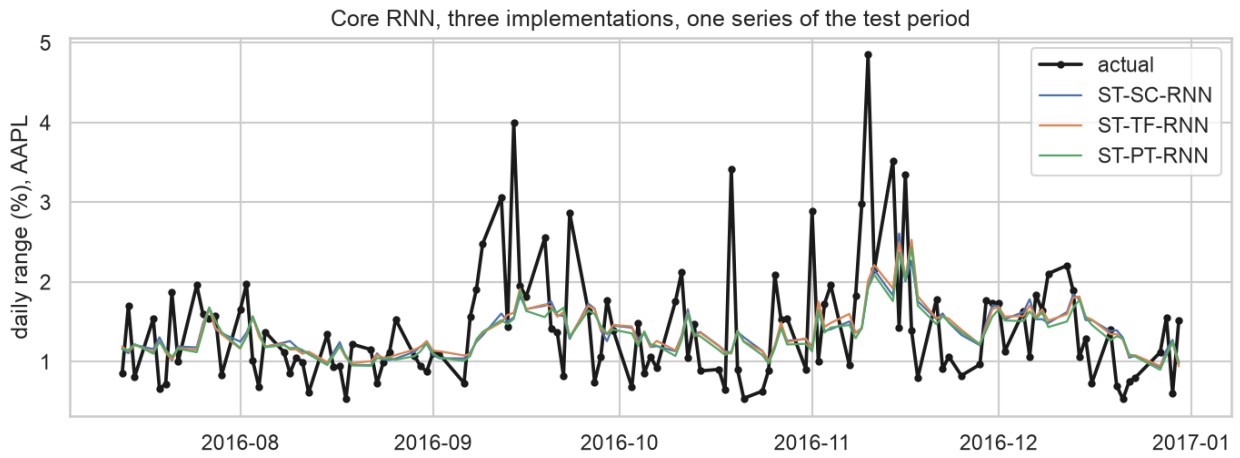


Figure 8: ST. Actual and predicted daily range of AAPL over the last 120 test days.

Findings.

- Risk is predictable: against persistence (1.39 pp, $R^2 = 0.28$) and the 5-day mean (1.16 pp, $R^2 = 0.47$) the RNNs reach $R^2 = 0.52$ – 0.54 and 70.5–71.1 % directional accuracy.
- HAR, with three coefficients, is as good: 1.145 pp, $R^2 = 0.520$, 70.0 %. Vanilla RNNs are slightly better on the z-scale they were trained on (R^2 0.522–0.529) and slightly worse in percentage points (1.18–1.19); the best model, the PyTorch LSTM, ties HAR (1.1454 vs. 1.1449). The network largely rediscovers a weighted average of recent volatility.
- The three core implementations agree closely (z-RMSE 0.747, 0.741, 0.742; all stop at epoch 3).
- Over 30 steps the vanishing gradient is visible: the oldest day receives 6 % of the last day's gradient after training and 0.1 % at initialisation (Figure 9). Gates close most of the gap to HAR (1.145–1.161 pp).

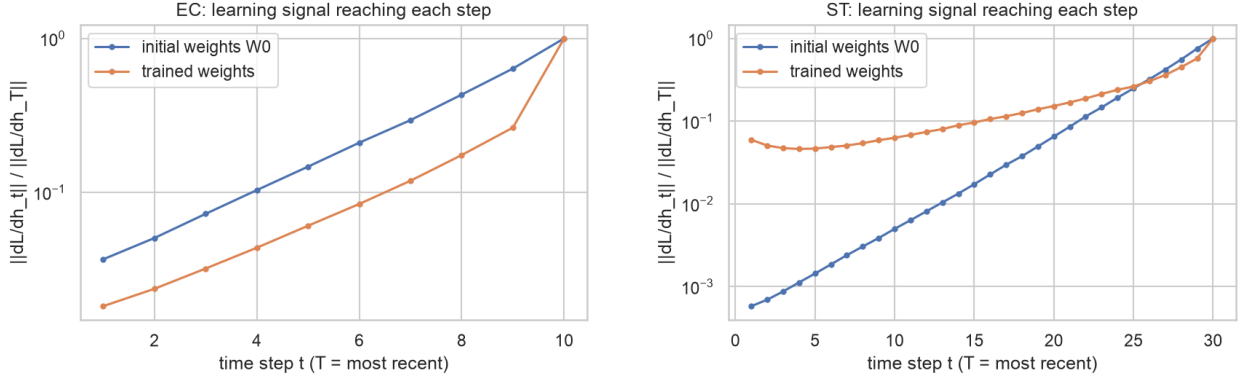


Figure 9: Gradient norm reaching each step of the window, relative to the last step, for the scratch RNN at initialisation and after training. Left: EC (10 orders). Right: ST (30 days).

8 Comparison of Implementations and Datasets

Same function, different abstraction. Scratch, Keras and PyTorch differ in how much of the loop the user writes: everything (scratch), nothing (Keras `fit`), or the loop but not the gradients (PyTorch). With the same starting weights they compute the same outputs; after training they differ by at most 2% in RMSE, and the mean absolute difference between their test predictions (0.05–0.09 z-units) is small compared with the prediction error (0.72–0.75).

Speed. For a 32-unit RNN the NumPy loop is the fastest trainer per epoch (EC 0.24s, ST 2.3s), Keras is 2.4–2.6× slower and PyTorch 3.3–3.5×: the matrices are too small for the frameworks’ per-batch overhead to pay off. PyTorch is among the fastest at inference (1.3ms per 1,000 EC windows). LSTM/GRU cost 2–3× the time per epoch of the vanilla RNN in the same framework.

Datasets. On EC the sequence of recent orders adds real information beyond any linear rule; on ST the signal is essentially a smoothed average of past volatility, which a linear model already captures. Gated cells help on both, more on the longer ST window where the vanilla RNN’s gradient vanishes.

9 Conclusions and Limitations

1. The RNN was implemented by hand (forward pass, BPTT, Adam, clipping) and verified twice: against finite differences ($\leq 10^{-9}$) and against Keras and PyTorch ($< 5 \cdot 10^{-7}$).
2. On e-commerce delivery times, recurrent models beat every baseline by 4–8% RMSE; on stock risk they match, but do not beat, the classic HAR model.
3. LSTM and GRU are the best models on both datasets, at several times the cost of the vanilla RNN.
4. The hand-written NumPy RNN is the fastest to train at this scale, while the frameworks make LSTM/GRU and larger models practical.

Limitations. One seed and one split per dataset, so differences below about 1–2% are not a ranking. EC has about 96k windows, below the 300k of assignment 05, because the dataset was limited to 100–150 MB and Olist has 99k orders. The EC split is by purchase time, so the May-2018 strike is only in validation. All timings are on one CPU.